Agent Traces with MLFLow and Red Hat OpenShift AI

Version 1, 2026-07-06

Home

Goal

This course teaches you how to deploy MLflow, as included with Red Hat OpenShift AI 3.4. You will learn how to collect traces of an agentic application using the MLflow SDK with Python. You will also learn how to visualize those traces using the Red Hat OpenShift AI web interface.

Learners should reserve a full working day to complete this course.

Pending review

Objectives

On completing this course, you should be able to:

- Understand how MLflow relates to agentic applications and to Red Hat OpenShift AI.
- Deploy a development environment with MLflow
- Collect traces from an agentic application using MLflow

Audience

- Software developers working on agentic applications and agentic harnesses.
- Red Hat and Partner roles that perform demonstrations, PoC and PoV of MLflow with Red Hat OpenShift AI, such as Solutions Architects.
- Red Hat and Partner roles that implement and support MLflow with Red Hat OpenShift AI, such as Services Consultants, Technical Account Managers, and Customer Support Engineers.

Prerequisites

This course assumes that you have the following experience:

- Familiarity with generative AI and agentic application concepts.
- Familiarity with the basic operation of Red Hat OpenShift AI.
- Python programming skills are recommended but not required, because this training provides sample code which is ready to run.

Classroom environment

The demo catalog entry Red Hat OpenShift AI 3.4 provides a single-node OpenShift cluster (SNO), backed by an AWS instance with a single GPU, with a predeployed `llama-32-3b-instruct` model which is capable of making tool calls.

This course uses that demo entry as-is. Learners will configure any additional components required

by either MLflow itself or any of the course activities, as part of course activities. A public Git repository contains all sample Python code and Kubernetes manifests required to complete hands-on activities.

Other sources of information

For documentation about MLflow with Red Hat OpenShift AI, see the Red Hat OpenShift AI product documentation.

For upstream MLflow documentation, see the MLflow project documentation, especially the section about Tracing (Observability).

Author

Fernando Lozano
Training Content Architect
Red Hat - Product Portfolio Marketing & Learning

Thanks to Adel Zaalouk, Andy Stoneberg, Calum Murray, Cansu Kavili Oernek, Francisco Arceo, Harshad Reddy Nalla, Jaideep Rao, Lazaro Coronado Torres, Matt Prahl, Peter Double, and Waldirio Pinheiro for their support creating this course and its hands-on activities.

Lab Environment

Instructions to Launch Your Lab on the Red Hat Demo Platform (RHDP)

1. Log in to the RHDP portal
2. Search for the catalog entry: `Red Hat Openshift AI 3`
3. Click the catalog entry in the search results.
4. On the catalog page, click **Order**.
5. Fill out the required details in the order form and accept the defaults for other fields. You can use the smaller AWS instance type.
6. Review the warning at the bottom of the form and check:
   I confirm that I understand the above warnings.
7. Await the email notification stating that your demo environment is ready and providing its access credentails and URLs.

Important Notes:

- This lab may take approximately **1:30** hours to become ready.
- You can also retrieve access information directly from the RHDP portal.
- This lab uses GPU-enabled instances on AWS, which are in short supply. It require multiple attempts to provision or restart.

Do **not** follow the instructions/showroom from the demo catalog entry. This course reuses that demo but does not require that you perform any of its activities.

You will use the preprovisioned instance of Red Hat OpenShift AI as-is, and deploy MLFlow and sample agentic applications as part of this course activities.

How to Access Your Lab via the RHDP Portal:

1. On the RHDP portal, click **Services** option in the left-hand menu.
2. Select your lab from the services listings on the right-hand side of the page to view access details.

RHDP Portal Links

- RedHat associates: https://demo.redhat.com/
- RedHat partners: https://partner.demo.redhat.com/

Understanding MLflow and Red Hat OpenShift AI

Goal

Understand how MLflow relates to agentic applications and to Red Hat OpenShift AI.

This chapter introduces MLflow in the context of Red Hat OpenShift AI, focusing on features relevant to generative AI and agentic applications. You'll learn about MLflow's architecture, supported features, and how it integrates with RHOAI components.

Introduction to MLflow in Red Hat OpenShift AI

Objective

Understand MLflow's features for agentic applications and its place as part of Red Hat OpenShift AI.

Work In Progress

Introduction to MLflow

According to their own web site:

> MLflow is the largest open source AI engineering platform for agents, LLMs, and ML models. MLflow enables teams of all sizes to debug, evaluate, monitor, and optimize production-quality AI applications while controlling costs and managing access to models and data.

MLflow provides fundamental observability and experiment tracking capabilities which enable developers and data scientists to ensure their applications work beyond a demonstration and perform as expected in production.

MLflow capabilities support developers and data scientists optimizing their applications for cost and accuracy, for example as they select smaller models or tuning their prompts, so they are confident they did not introduce unacceptable performance degradation.

It is a known fact that AI-enabled applications usually degrade over time, because user behavior and input data changes. MLflow enables continuous evaluation of runtime performance of applications, so improvements can be made before users start complaining.

High-level architecture of MLflow

MLflow is based on a client/server architecture. A client application, which is instrumented by using either the MLflow SDK or OpenTelemetry, sends observability data, such as metrics and traces, to an MLflow server, which stores that data and provides HTML-based visualization of it.

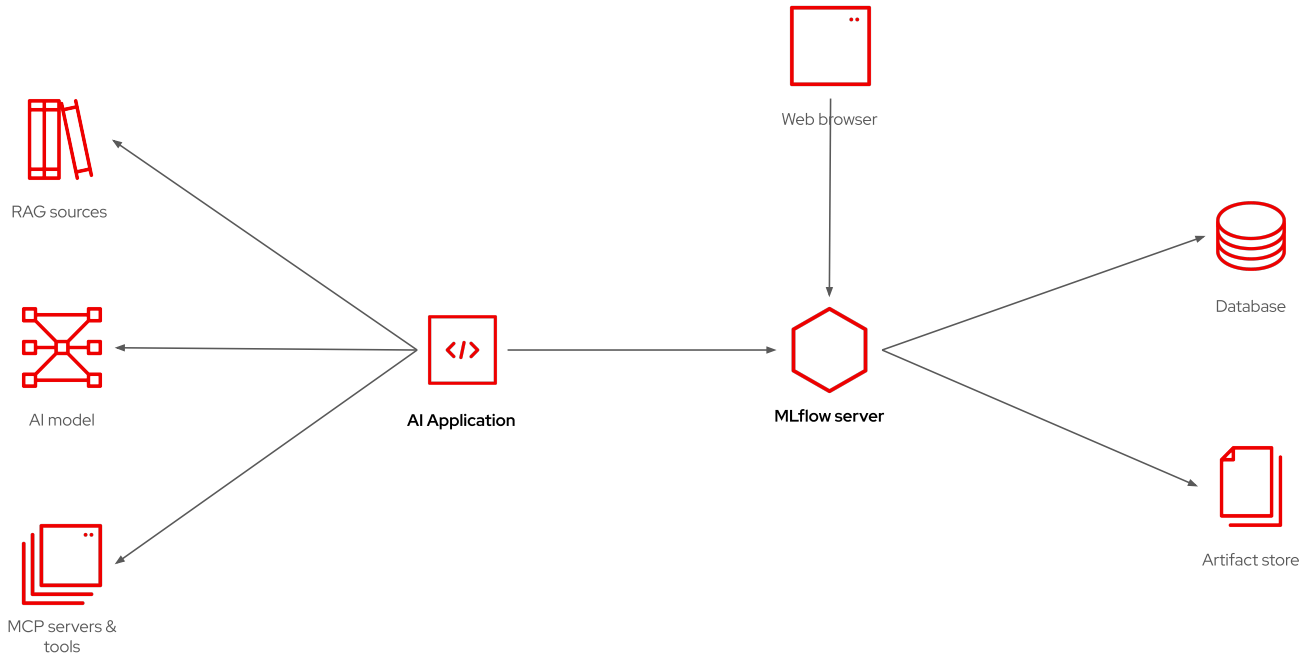

Figure 1. Architecture of MLflow

The application is responsible for interacting with ML models or, in the case of GenAI applications, with LLMs. The GenAI application is also responsible for interacting with MCP servers, providing local tools, or connecting to RAG data sources.

It does not matter whether the "application" is a user-facing application, an autonomous agent, or a training or evaluation pipeline. It just needs to send observability data to an MLflow server.

Client applications can also consume data from MLflow servers, for example to compare metrics from different training sessions and determine which one provided better outputs, or to track performance degradation, over time, for a production application.

The MLflow server makes use of a relational database and an artifact store to record artifacts such as model weights (in case of training ML models) or prompts (in case of GenAI applications). It is recommended that the artifact store be an S3 service, for scalability.

The relational database stores metadata and tags which enables the MLflow server to relate all inputs, outputs, and observability data from a single training session or from a single conversation with an LLM to each other. A PostgreSQL database is a popular option for the MLflow database.

MLflow for Generative AI Applications

MLflow started by supporting Machine Learning (ML) applications and data scientists, offering features to support model training and tuning. Like many tools focused on ML, MLflow was initially very Python-centric.

More recently, MLflow added features for supporting Generative AI (GenAI) applications and agents. By that time, MLflow was also offering native SDKs for other programming languages, such as TypeScript and Java.

Nowadays, MLflow also supports the OpenTelemetry standard, which enables applications using any programming language and any kind of infrastructure to submit observability data to an MLflow server.

MLflow experiments and workspaces

Experiments are a key concept in MLflow. An *experiment* is a grouping of observability data, from multiple runs. Each run collects all data obtained while running an AI application once. Runs may take multiple forms, and the ones we are interested in for this course are *traces*, which record the interactions between an agent and a model.

A workspace is a grouping of experiments. Typically, a workspace corresponds to an application, and experiments correspond to running the application with different configuration parameters, such as different models, or different system prompts.

Then, inside an experiment, each run connects an input to its output, and provides metrics and other observability data from that run. Frequently there are multiple runs, such as multiple traces, which started from exactly the same input data because LLMs are inherently non-deterministic.

MLflow Features in Red Hat OpenShift AI

MLflow is a comprehensive platform for managing AI applications (AIOps) but it does not provide *everything* data scientists, software developers, and AI engineers need to develop and manage their ML or GenAI applications. Providing "everything" is the stated goal of Red Hat OpenShift AI (RHOAI). While MLflow provides many features which are useful by themselves, others work better when paired with tools outside of MLflow.

Sometimes, there's overlap between a feature from MLflow and a feature from another component of RHOAI, for example the Model Registry which in RHOAI comes from KubeFlow. All features from MLflow are available from RHOAI but not all of them may be integrated with the RHOAI dashboard and with other features of RHOAI.

The upstream web interface from MLflow is available unchanged, as a stand-alone application, from RHOAI. Users are free to continue using this interface. But you may find it is easier and more convenient to use the subset of features made available from RHOAI dashboard, because it facilitates using these features integrated with other components and features from RHOAI.

What's Next

Now that you understand MLflow's role in RHOAI and agentic applications, the next activity walks through the environment for hands-on activities, which provides an instance of RHOAI preconfigured with an LLM.

Lab: Explore the RHOAI Instance

Objective

Explore the predeployed Red Hat OpenShift AI instance to familiarize yourself with its configuration and with the new Gen AI Studio feature from 3.x releases.

Pending review

Before you begin

You need access to the predeployed Red Hat OpenShift AI (RHOAI) instance provided in your lab environment. Ensure you have:

- Access credentials for the RHOAI dashboard
- A web browser to access the RHOAI dashboard

Instructions

In this lab, you will log in to the RHOAI dashboard and explore its main features and components to understand the platform's capabilities.

Inspect the existing projects and deployed models

1. Access the RHOAI Dashboard.
   a. Navigate to the RHOAI dashboard URL of your demo environment.
   b. Log in using the `admin` user, from the credentials provided for your demo environment.
2. Explore the precreated project.

   Your RHOAI instance contains a model which is already deployed in a regular project named `my-first-model` — which is **not** an AI project.

   That model takes up all GPU capacity available to your RHOAI instance, so any change to the model deployment requires that you stop it and restart after making changes.

   a. In the left pane, select **AI hub › Models** and then select the **Deployments** tab.
   b. In the **Project** field, type `my` and select the **my-first-model** project.

   

   Depending on the current page of the RHOAI dashboard, it may be necessary to first select `All projects` before you can search and select the `my-first-model` project.

   c. Verify that there is a model deployment named `llama-32-3b-instruct` and its status is `Ready`.
   d. Click **View**, under the **Endpoints** column for the model row, and verify that it provides only an internal endpoint. This means the model cannot be accessed from outside of its OpenShift cluster, unless you change its model deployment.
3. Try the model with a Gen AI Studio Playground.
   a. In the left pane, select **Gen AI studio › Playground**.
   b. In the **Project** field, type `my` and select the **my-first-model** project.

   

   The project previously selected in one page of the RHOAI dashboard may not be the current selection in another page of the RHOAI dashboard.

   c. If you do not see a **Configure** pane, click **Settings**. In the **Configure** pane, select the **Model**

tab and verify that the selected model is named `llama-32-3b-instruct`. Ignore the prefix on the model name.

d. In the right pane, click the `Send a message⋯` text and type a prompt, for example:

```
What is RHEL?
```

e. Type `Enter` to submit the prompt and see the response from the model. The model may return different answers if you attempt the same prompt again, but they all would, hopefully, be correct and reasonable.

Confirm that MLflow is not deployed yet in RHOAI.

1. Verify that there are no RHOAI dashboard pages related to MLflow.

   a. In the left pane, expand the **Gen AI studio** category, which contains entries named **Playground** and **AI asset endpoints**.

   After you enable MLflow, there will be additional entries here.

   b. Select **Gen AI studio › AI asset endpoints** and then select the **MCP servers** tab.

   Verify that there are no MCP servers configured for use by Gen AI studio playgrounds.

   c. In the left pane, expand the **Develop & train** category, which contains entries named **Feature store**, **Pipelines**, **Jobs**, and **Evaluations**.

   d. In the toolbar, to the top right, click the **Applications** icon, which is close to the **Question** icon, and verify that there is **not** an entry named **MLflow**.

Summary

After completing these steps, you have verified that you have a functional RHOAI instance with a pre-deployed model, which is available only for internal access, and that you can submit prompts to the model using an RHOAI playground.

In addition, you have also verified that MLflow is not installed on your RHOAI instance, and that there are no MCP servers deployed.

What's next

In the next chapter, you will learn how to deploy MLflow on Red Hat OpenShift AI to enable experiment tracking and model management capabilities.

Deploying MLflow for Development

Goal

Deploy a development environment with MLflow

This chapter guides you through deploying MLflow in Red Hat OpenShift AI for development purposes. You will learn about MLflow's architecture, how it integrates with RHOAI, and gain hands-on experience deploying and verifying your MLflow instance.

MLflow within Red Hat OpenShift AI

Objective

Understand how MLflow integrates with Red Hat OpenShift AI.

Work In Progress.

The MLflow Operator and RHOAI

There's an MLflow operator included with Red Hat OpenShift AI (RHOAI), but this operator won't appear in the software catalog or in the list of installed operators managed by the Operator Lifecycle Manager (OLM). The MLflow operator is managed by the RHOAI operator itself.

To deploy an MLflow server, you must perform two actions:

1. Edit the singleton data science cluster resource of your RHOAI instance to enable the MLflow operator by setting its `spec.components.mlflowoperator.managementState` field to `Managed`.
2. Create an MLflow resource, which configures the database and artifact storage of the MLflow server.

Once the MLflow resource is available, the RHOAI dashboard displays a number of new pages, or adds more elements to some of its existing pages.

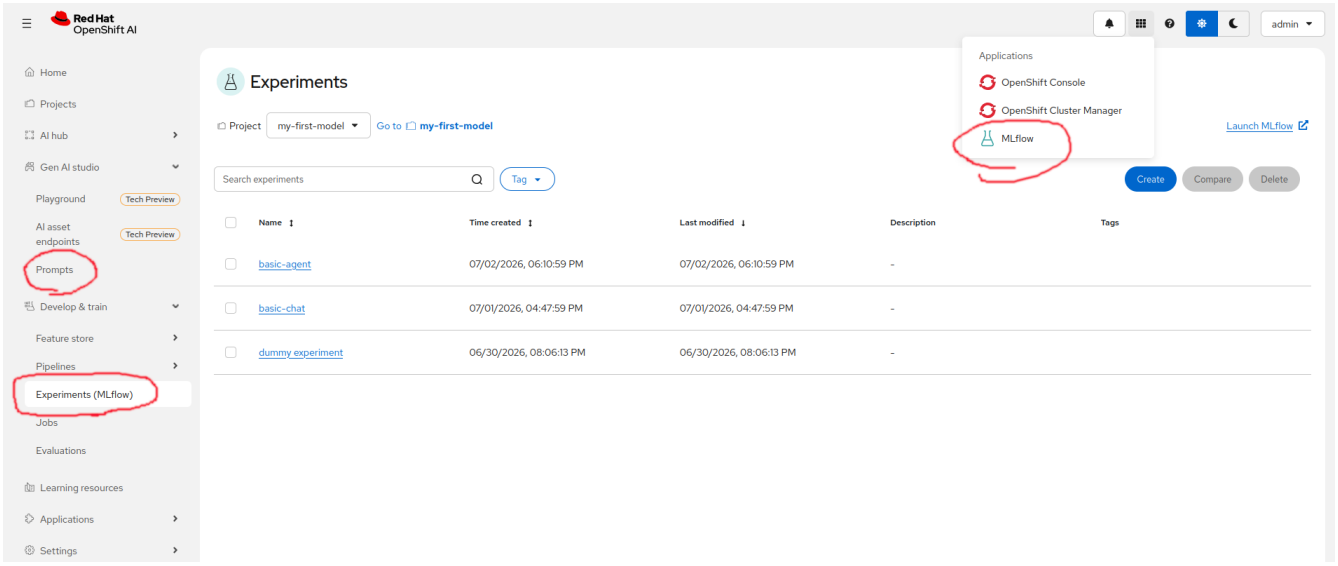

Figure 2. RHOAI dashboard in the Experiments page, and new pages from deploying an MLflow server

The experiments page, shown by the previous screenshot, is your main user interface to MLflow in RHOAI 3.4. Future releases may provide additional UI integrations, for example in the GenAI studio.

You may notice that some RHOAI dashboard pages are actually iframes which display the body of a page from the MLflow web UI. It is a very light integration, and Red Hat is an active contributor to the upstream MLflow project.

MLflow workspaces and authentication within OpenShift

The MLflow operator maps Kubernetes namespaces and OpenShift projects (not just AI projects) to MLflow workspaces. That is, for each project in RHOAI there is an MLflow workspace.

The MLflow operator also configures the MLflow server to delegate authentication and authorization to OpenShift, so any OpenShift user is a valid MLflow user. Such users have access to data in MLflow workspaces based on their RBAC permissions from the respective OpenShift project.

Besides, service accounts with access to resources in a namespace automatically gain access to the MLflow workspace. This means pods running in OpenShift, including GenAI studio playgrounds and AI workbenches (including their Jupyter notebooks) automatically get access to MLflow workspaces.

In summary, you manage credentials and permissions to access the MLflow server managed by RHOAI by configuring OpenShift users and standard Kubernetes RBAC. For example:

- You can enable applications to submit observability data by granting them the `edit` cluster role on their projects.
- You can enable users to view (but not change) observability data by granting the `view` cluster role on the application project.

Older releases of the MLflow SDK did not include the `kubernetes` extension, which automatically retrieves the service account token from the running pod and uses it as the MLflow authentication token. That required using an RHOAI fork of the MLflow SDK, but that extension is already merged upstream since MLflow 3.10.

If you must connect to MLflow an application running outside of its OpenShift cluster, you can take same user token you would use for access to Kubernetes APIs and use it to authenticate to the MLflow server. External applications connect to the MLflow server using the same route (URL) a web browser uses to access the stand-alone MLflow web UI.

Development vs production deployments of MLflow

RHOAI product documentation provides two examples of MLflow resources, one for a development environment, and another for a production environment.

- The development example runs an in-container SQLite database, which stores data on a PVC attached to the MLflow server pod, and uses the same PVC as its artifact store. You can increase the size of that PVC and you are responsible for backing up data from it, possibly using the OADP API.

- The production example uses an external PostgreSQL database server and an external S3 object storage service, for example AWS S3 or OpenShift Data Foundations (ODF) from Red Hat OpenShift Plus. You are responsible for providing access credentials and for managing high availability, disaster recovery, and data retention of both the external database and S3 object buckets.

Nothing prevents you from using a managed database service from a cloud provider as your MLflow database. Just be aware that RHOAI and its MLflow operator do not manage the relational database and object storage in any way. They just store data there.

What's Next

Now that you understand MLflow's architecture and its integration with RHOAI, the next section provides hands-on experience deploying MLflow in a development environment.

Lab: Deploy MLflow on Red Hat OpenShift AI

Objective

Deploy and verify a development instance of MLflow in Red Hat OpenShift AI.

Pending review

Before you begin

You need access to the predeployed Red Hat OpenShift AI (RHOAI) instance provided in your lab environment. Ensure you have:

- Access credentials for the RHOAI dashboard
- A web browser to access the RHOAI dashboard
- Access credentials for the OpenShift web console

Instructions

You will enable MLflow on your Data Science Cluster singleton resource and deploy a development instance of the MLflow server. Then you will create a simple Jupyter notebook to test connectivity to the MLflow server. You will use the MLflow SDK for Python.

As you perform this activity, you will alternate between three browser tabs:

- The Red Hat OpenShift web console
- The Red Hat OpenShift AI (RHOAI) dashboard
- A Jupyter notebook from a RHOAI workbench.

Once you open one of these tabs, keep it open because you may return to it later.

Install and configure the MLflow Server

1. Enable MLflow on RHOAI by editing its data science cluster resource.
   a. Open the OpenShift web console and log in as a user with `cluster-admin` rights.
   b. In the left navigation pane, select **Ecosystem › Installed Operators**. Then click **Red Hat OpenShift AI**.
   c. In the Red Hat OpenShift AI operator page, select the **Data Science Cluster** tab. Then click the **default-dsc** resource.
   d. In the `default-dsc` resource page, select the **YAML** tab. In the text editor, search for the `spec.components` field, and set its `mlflowoperator.managementState` field to `true`.

```
...
spec:
  components:
    mlflowoperator:
      managementState: Managed
    kserve:
      managementState: Managed
      rawDeploymentServiceConfig: Headless
...
```

   e. Click **Save** and wait until you see a confirmation message similar to:

```
alert:default-dsc has been updated to version 1186240
```

   f. If you get an error message, fix the errors on your YAML and try saving again.
   g. Alternatively, you could run the following `oc patch` command, which is provided by the RHOAI product docs:

```
$ oc patch datasciencecluster default-dsc --type=merge \
-p '{"spec":{"components":{"mlflowoperator":{"managementState":"Managed"}}}}'
datasciencecluster.datasciencecluster.opendatahub.io/default-dsc patched
```

2. Configure your MLflow instance by creating an MLflow resource.
   a. In the left navigation pane, select **Home › API Explorer**. Then type `MLflow` in the search field. You will see three resource types, click on **MLflow**.
   b. In the MLflow Resource details page, select the **Instances** tab and click **Create MLflow**.
   c. In the YAML text editor, delete all of its sample contents and type the following, which corresponds to the example from Minimal development or test deployment in product docs.

```
apiVersion: mlflow.opendatahub.io/v1
kind: MLflow
metadata:
```

```
  name: mlflow
  namespace: redhat-ods-applications
spec:
  storage:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: 10Gi
  backendStoreUri: "sqlite:////mlflow/mlflow.db"
  artifactsDestination: "file:///mlflow/artifacts"
  serveArtifacts: true
```

d. Click **Create** and, in the MLflow details page, wait until the `Available` condition is `True` and the `Progressing` condition is `False`.

Test your MLflow server using the dashboard and a workbench

1. Verify MLflow is available from RHOAI and create an empty experiment.
   a. Open your RHOAI dashboard. If it is already open, refresh your browser window.
   b. In the left navigation pane, expand **Develop & train**. Notice the new item named **Experiments (mlflow)** and click it.
   c. In the **Project** field, select the **my-first-model** project.
   d. Click **Create Experiment** and type `dummy experiment` as its name.
   e. Verify that you see the Experiments page for `dummy experiment` but there is no data on any of its pages, such as **Traces**, **Sessions**, or **Prompts**.
   f. Creating an empty experiment is sufficient to prove that your MLflow instance can connect to its database and artifacts storage.
   g. You can, alternatively, open the upstream MLflow web interface by clicking the applications menu, in the top navigation bar, close to the help icon, and selecting **MLflow**. You should not need it: all functionality you need for this course is available from the RHOAI dashboard.
2. Test connectivity to MLflow from a Jupyter notebook inside a workbench.

If you have trouble typing the Python code in this and other activities of this course, or you cannot cut-and-pate from the course page, you can access the complete source code for all jupyter notebooks in a git repository.

   a. In the left navigation pane, click **Projects** and select **All projects** in the projects combo box. Then type `my` in the filter by name field and click **my-first-model** in the list of projects.
   b. In the Projects page for `my-first-model`, select the **Workbenches** tab and click **Create workbench**.
   c. In the Create workbench page, type `test-mlflow` for the **Name** field and, in the **Workbench image** field, select **Jupyter | Minimal | CPU**.
   d. Leave all other fields with their default values and click **Create workbench**.

e. In the Workbenches list, wait until the row for the `test-mlflow` workbench displays a `Ready` state and then click **test-mlflow** to open its Jupyter Lab in a new browser tab.

f. Click the **Python 3.12** Notebook icon to create an empty notebook.

g. In the first cell, type:

```
! pip install mlflow[kubernetes]
```

h. In the top toolbar from the notebook, click the **play** icon, with the tooltip "Run this cell and advance".

i. Wait until the MLflow SDK and all its dependencies are loaded from the RHOAI index into the workbench.

j. After the `pip` command finishes, type the following on the second cell of your notebook:

```
import os
import mlflow

os.environ["MLFLOW_TRACKING_AUTH"]="kubernetes-namespaced"
mlflow.set_tracking_uri("CHANGE_ME")

print(mlflow.list_workspaces())
```

k. Switch to your RHOAI dashboard page, which should still be in the Experiments page. Notice, close to the top left, the **Launch MLflow** link. Right-click it and copy its URL to your clipboard.

l. Switch to your Jupyter Lab browser tab and replace, in the second notebook cell, the `CHANGE_ME` text with the URL on your clipboard.

In this example, we are using the *external* URL of the MLflow server, which depends on the applications domain of your OpenShift Ingress operator. This is the URL you would use for applications running outside of its OpenShift cluster.

External applications also need to set the `MLFLOW_TRACKING_TOKEN` environment variable with a valid OpenShift authentication token, for example the one returned by the `oc whoami -t` command.

m. In the top toolbar from the notebook, click the **play** icon. The results should be an array containing a single `Workspace` object named `my-first-model`.

This notebook takes advantage of the integration between the RHOAI dashboard and its managed instance of the MLflow server. It connects using the service account of the `test-mlflow` workbench, which is authorized to access the MLflow workspace corresponding to its `my-first-model` project.

n. If you wish, you can remove the unnamed notebook, or if you prefer you can save it, rename

it to `test.ipynb`, and reuse it for the next activity. Whatever your preference, do NOT delete the `test-mlflow` workbench because you will keep using it in the next activity.

Summary

After completing these steps, you have verified that you have a minimal but functional MLflow instance, to which you can connect from applications running inside your RHOAI cluster and store data from your experiments.

What's next

Now that you have a working MLflow deployment, the next chapter explores how to collect traces from agentic applications using the MLflow SDK.

Collecting Traces from Agentic Applications

Goal

Collect traces from an agentic application using MLflow

This chapter teaches you how to use the MLflow SDK to collect traces from agentic applications. You'll learn why tracing is essential for troubleshooting agents, how to organize traces in experiments and runs, and gain hands-on experience adding MLFlow tracing to your code.

Tracing Agentic Applications

Objective

Understand why tracing is essential for agentic applications and how MLflow organizes trace data.

Work In Progress

Why developers need traces

If you provide exactly the same data to an LLM, in multiple attempts, each one returns a different response. Sometimes, it's a small difference in wording. But sometimes it is a completely non-sense response, that is, an hallucination.

Developers working on agentic applications want to reach a compromise between the accuracy and the cost of running the agent. The accuracy is the number of good or correct answers compared to the number of bad or incorrect answers, by whatever criteria you define "good" and "bad".

You never reach 100% accuracy, but there are approaches to handle the inevitable bad answers, such as adding guardrails and evals before displaying the answer to an end user or before using the answer for the next task in an automated workflow.

The cost of running an agent depends mainly on the token usage, and again each attempt, even if taking exactly the same input data, may have a different cost. Larger models, reasoning models, and frontier models provide a higher accuracy at a higher cost per token and by spending a larger amount of tokens per request.

So you must run your tests many times using the same input data. And you also must run tests using different input data: different kinds of requests which you expect that your agent should be able to answer or process.

As you attempt to optimize your agent (or just reach a good enough working state) you will also try variations on parameters such as system prompts, tool sets, and RAG sources. Maybe even try with different models.

In fact, collecting traces of an agent is the start of a process which requires post-processing of the data in such traces. RHOAI provides tools for such post-processing, for example the **Eval Hub** feature, which are **not** in scope for this course.

Traces and spans

MLflow traces are useful for both interactive applications and autonomous agents. They collect all inputs and outputs of each individual interaction between an application and a model, for example:

- The model name
- The system prompt
- Model invocation parameters, such as temperature
- The final response from the model
- Error messages from the model and/or from the application, such as context exhaustion

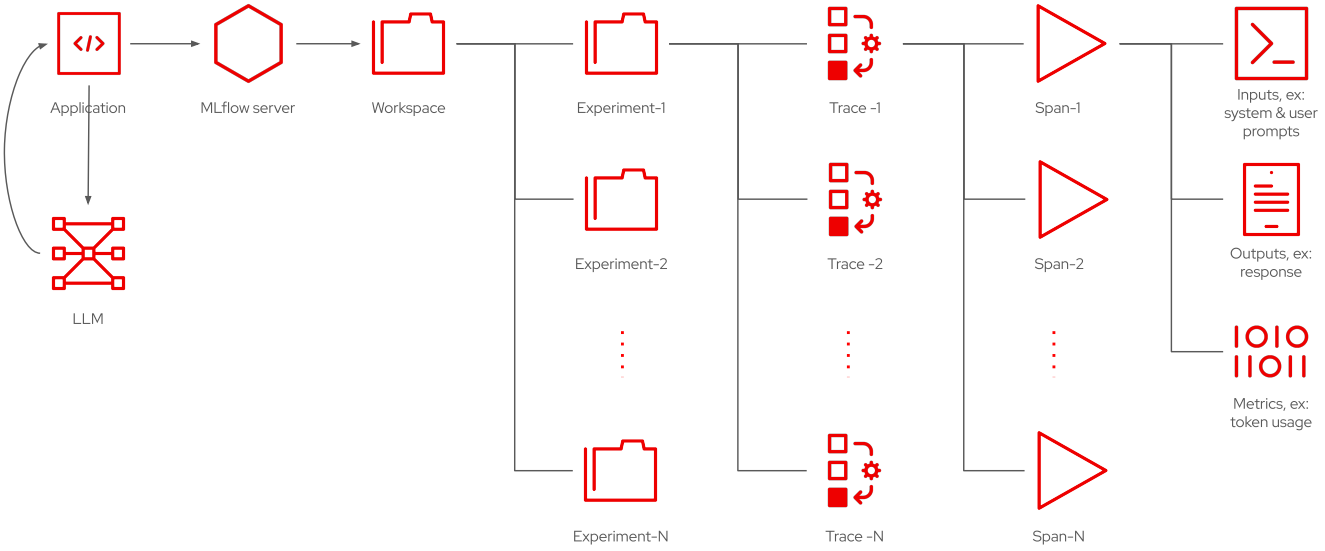

Figure 3. Workplaces, experiments, traces, spans, and data recorded for each span

Every time you run the application again (or submit a new prompt) it generates a new trace. If it is a conversational application, each subsequent prompt from the same conversation is added to the same trace, as a new *span*, until the conversation is terminated.

If your application makes use of RAG sources, it should include in the trace all data returned by RAG, which is added to the context of the model, as additional inputs.

From a trace, it should be possible to reproduce the conversation, providing exactly the same context, and compare the final and intermediate responses, in addition to metrics such as latency and token usage.

Traces and MCP servers

If it is an agentic application, the response from the model may request one or more tool calls. These tools are usually provided by one or more MCP servers. Agents are expected to provide a model with a list of all tools they are able to call, and that for each and every request!

The agent is responsible for performing all tool calls, collecting their outputs, and adding these outputs to the next request submitted to the model, as part of the same conversation.

The model may decide to request yet additional tool calls, until it is satisfied it has collected all data it needs to provide a final response. It is like the agent and the model are having a conversation

between themselves, without any participation from the end user.

An agent runs a loop, checking if the model requests another tool call, and only when the model signals its done the agent displays the final response (or takes whatever action it does from the final response).

As the agent runs its loop, its current trace grows with more spans. Each span records the current state of the context, including prompts, available tools, previous messages and previous responses.

The label you see on your spans depend on the client framework and on the usage of auto-trace or manual trace. So they may vary between applications and frameworks. Anyway, be ready to look at a lot of hierarchical data inside a trace.

Viewing traces with the RHOAI dashboard

RHOAI displays traces for the selected project in the **Develop & train › Experiments** page.

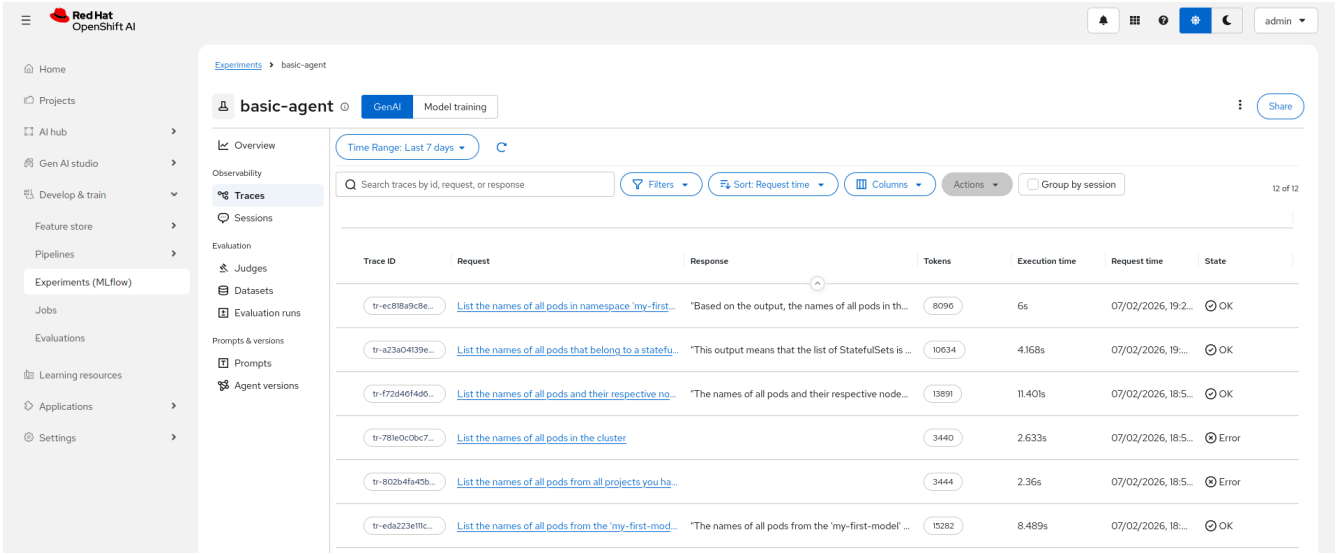

Figure 4. List of traces in an experiment

Each trace gets an auto-assigned ID. Click either the ID or the user prompt (in the **Request** column) to view the spans inside that trace.

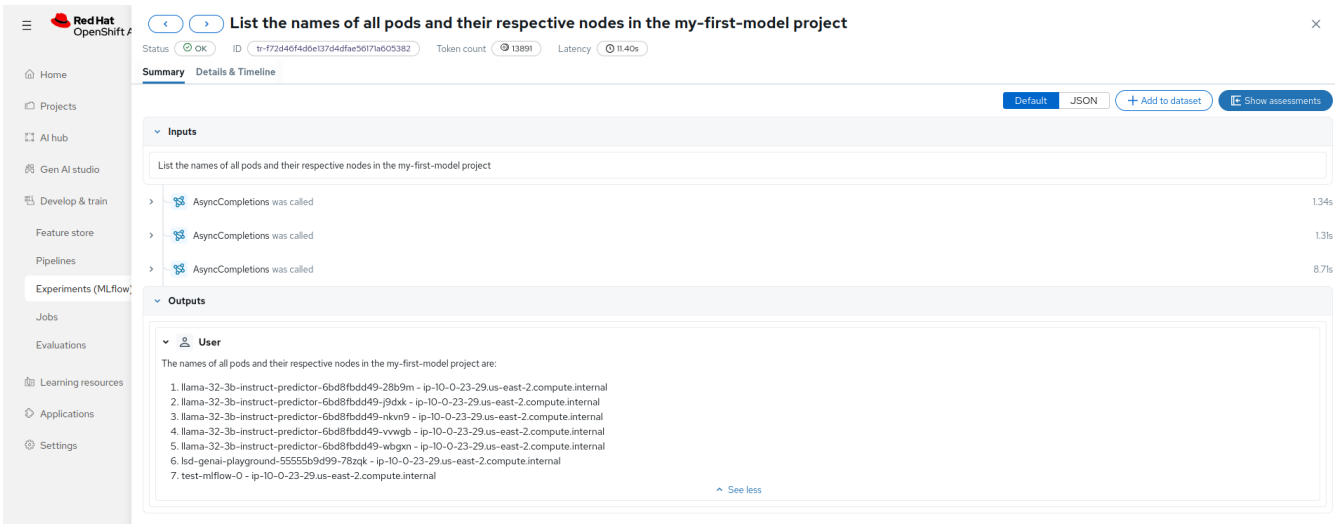

Figure 5. List of spans inside a trace

In the previous figure, each span is labeled `AsyncCompletions` because of the OpenAI extension for agents.

You can expand each individual span to show its input, outputs, and other data:

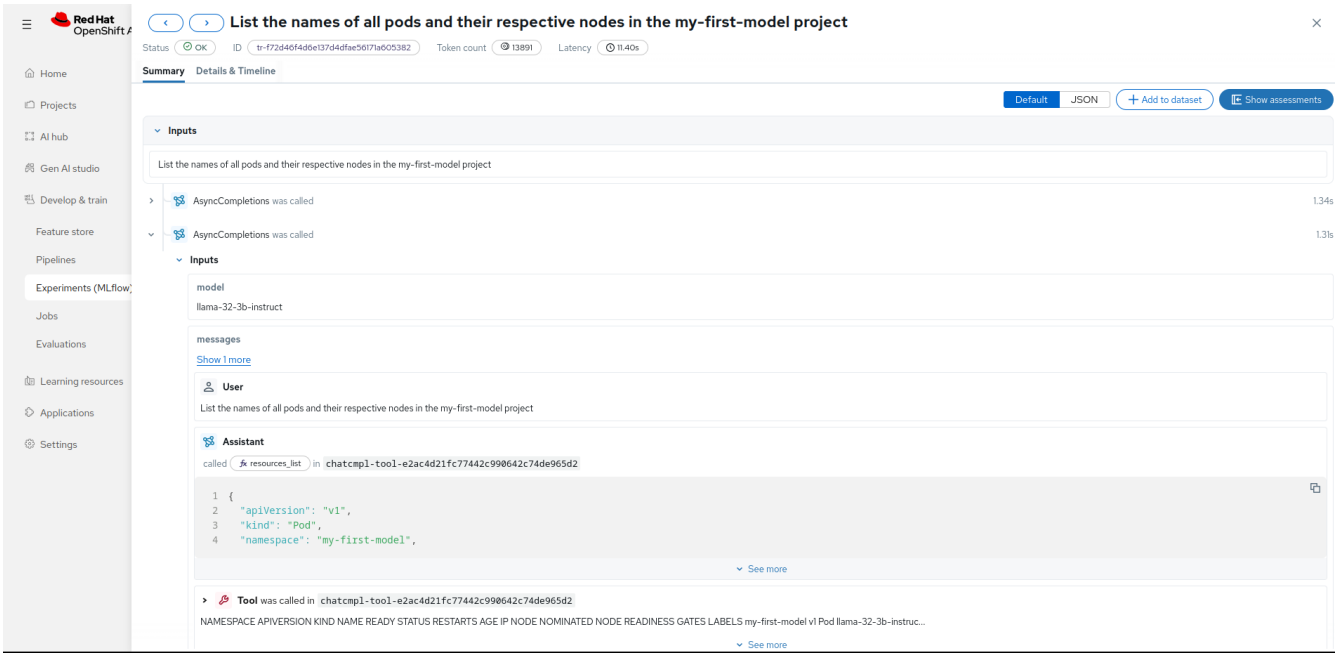

Figure 6. A span that submits the data from a tool call

The previous example shows a span where the agent provides to the model data returned by a tool call, which was requested by the model during the previous span.

Because the context in a conversation with a model is cumulative, it is usually sufficient to check the latest span in a trace. This is true for simple agents.

More sophisticated agents could orchestrate a complex workflow, which submits requests to different models. They can also control how much context to submit for each request. In such cases, as well as with conversational applications, you have the option of organizing related traces into *sessions*.

This course only presents very simplistic sample applications, which are not conversational and do not delimit sessions.

Tracing with the MLflow SDK

The simplest way of generating tracing from your agent, at least with the MLflow Python SDK, is to use its *autologging* feature.

For example, if you application uses OpenAI APIs:

```
mlflow.openai.autolog()
```

If your agent runs as a pod inside OpenShift, its project is automatically taken as its MLflow workspace. But you can and should set the name of the current experiment, **before** you start

logging traces.

```
mlflow.set_experiment("my-experiment")
```

This course will not detail most of the tracing functions available in the MLflow SDK. If you must, you can have fine control of when you start a new span, how you label spans. Similarly, you can also control when you start and stop sessions and how you label sessions. You can also append additional parameters and metrics to each span.

It is also possible to use a generic logging framework, if it is configurable as an OpenTelemetry provider. These and other alternatives are detailed in the MLflow community documentation.

What's Next

Now that you understand tracing concepts, the next section provides hands-on experience modifying an agent to record traces using the MLflow SDK. First, you add tracing to a basic chat application. Then, you run an agent which makes MCP tool calls and inspect its traces.

Lab: Modify an Agent to Record Traces

Objective

Add MLflow tracing to an agentic application and visualize the generated traces.

Pending review

Before you begin

You need access to the predeployed Red Hat OpenShift AI (RHOAI) instance provided in your lab environment, which was already configured by the previous lab with a development instance of the MLflow server and an RHOAI workbench.

Ensure you have:

- Access credentials for the RHOAI dashboard
- A web browser to access the RHOAI dashboard
- Access credentials for the OpenShift web console

Instructions

You will create a simple chat application using a workbench, validate it is working, and instrument the application with calls to MLflow. Then, you will review traces generated by a couple test runs of the instrumented application.

As you perform this activity, you will alternate between three browser tabs:

- The Red Hat OpenShift web console

- The Red Hat OpenShift AI (RHOAI) dashboard
- A Jupyter notebook from a RHOAI workbench.

Once you open one of these tabs, keep it open because you may return to it later.

Create a simple chat application

1. Create a Jupyter notebook for your chat application.
   a. Open the RHOAI dashboard and log in using the `admin` user.

   

   You do not need administrator privileges to complete this lab, but the demo instance is not configured with any user other than `admin`.

   b. In the left navigation pane, select **Projects** and, in the **Project** field, select **All projects**. Then type `my` and, in the list of projects, select **my-first-model**.
   c. In the projects page for `my-first-model`, select the **Workbenches** tab. Click the **test-mlflow** workbench, which you created in the previous lab, to open its jupyter lab in a new browser tab.
   d. In the Jupyter Lab, select the **Launcher** tab and click the **Python 3.12** icon to create a new Jupyter notebook.
   e. In the left navigation pane, right click the **Untitled.ipynb** file, which is your new notebook, and select **Rename**. Then type `basic-chat` as its new file name.
   f. Alternatively, you can open and reuse the `test.ipnyb`, which you created in the previous lab.
2. Insert the code for a simple chat application into your notebook.
   a. In the first cell, use a `pip` command to install the OpenAI client API:

   ```
   ! pip install openai
   ```

   b. In the top toolbar from the notebook, click the **play** icon, having as tooltip "Run this cell and advance".
   c. Wait until the OpenAI library and its dependencies are loaded from the RHOAI index into the workbench.
   d. In the second cell, set some variables designed to adapt your application to its running environment.

   ```
   BASE_URL = "CHANGE_ME"
   model_name = "llama-32-3b-instruct"
   ```

   e. Switch to the RHOAI dashboard and select, in the left navigation pane, **AI hub › Models**, then select the **Deployments** tab. In the row for the `llama-32-3b-instruct` model, click **Internal endpoint** to display a pop-up with the access URL for its model server, and click the **Copy** icon in the pop-up.

f. Switch to your Jupyter notebook and replace the `CHANGE_ME` text with the URL on your clipboard and type `/v1` at the end of the URL to make it a valid OpenAI API endpoint.

g. Continue editing the second cell and set a system and a user prompt.

```
input_text = """
What's the difference between RHEL and CentOS?
"""
system_instructions = """
You are a helpful AI assistant.
You are designed to answer questions in a concise and professional manner.
"""
```

h. Add to the cell code to retrieve the authentication token from its Kubernetes service account and connect to the model server using the OpenAI API.

```
import os
from openai import OpenAI

try:
    with open("/var/run/secrets/kubernetes.io/serviceaccount/token", "r") as f:
        AUTH_TOKEN = f.read().strip()
except FileNotFoundError:
    # Fallback if running outside a workbench or using a manually generated
cluster token
    AUTH_TOKEN = OCP_TOKEN

client = OpenAI(
    base_url=BASE_URL,
    api_key=AUTH_TOKEN
)
```

i. Add code to submit the prompt to the model and display its response.

```
try:
    response = client.chat.completions.create(
        model=model_name,
        messages=[
            {"role": "system", "content": system_instructions},
            {"role": "user", "content": input_text}
        ],
        max_tokens=256,
        temperature=0.7
    )

    # Print the output
    print(response.choices[0].message.content)
```

```
except Exception as e:
    print(f"An error occurred: {e}")
```

3. Run your chat application to verify it is working.

   a. It is a good time to save your code changes. In the top toolbar of your notebook, click the **disk** icon, having as tooltip "Save and create checkpoint".

   b. Now that your second notebook cell contains the entire code of the chat application, click the **play** icon in the top toolbar of your notebook.

   c. Hopefully, your output is correct and useful. You do know what makes RHEL different than CentOS, right? If you get a hallucinated response, just continue with the activity. You will have a chance of seeing a good response later in this lab.

      Following is an example of a good response we got while testing these instructions:

```
Red Hat Enterprise Linux (RHEL) and CentOS are both popular Linux distributions,
but they have distinct differences:

1. Origin:
   - RHEL is a proprietary Linux distribution developed and maintained by Red
Hat, Inc.
   - CentOS is a community-supported Linux distribution, based on RHEL, and is
also known as "Community Edition" or "Free Community Supported Edition".

2. Support and Licensing:
   - RHEL has paid support and licensing, which provides access to Red Hat's
customer support, updates, and security patches.
   - CentOS is free and open-source, but it relies on the community for support
and updates.

3. Update Model:
   - RHEL follows a traditional release model, where new versions are released
every 2-3 years, and updates are provided through subscription.
   - CentOS follows a rolling release model, where new versions are released
based on the RHEL version, and updates are provided through the community.

4. Binary Compatibility:
   - RHEL and CentOS are binary compatible, meaning that software compiled for
RHEL will run on CentOS, and vice versa.

In summary, RHEL is a paid, enterprise-focused distribution, while CentOS is a
free, community-supported distribution based on RHEL.
```

Add tracing to the chat application

1. Instrument your chat application with MLflow tracing.

   a. Edit the first cell to also install the MLFlow SDK for Python.

```
! pip install openai mlflow[kubernetes]
```

b. Click the **play** icon in the toolbar and wait until the MLflow library and its dependencies are loaded from the RHOAI index into your workbench.

c. Edit the second cell to add a few variables, which point to the MLFlow server from your RHOAI instance.

```
BASE_URL = "http://llama-32-3b-instruct-predictor.my-first-
model.svc.cluster.local:8080/v1"
model_name = "llama-32-3b-instruct"

MLFLOW_TRACKING_URL = "https://mlflow.redhat-ods-
applications.svc.cluster.local:8443"
MLFLOW_EXPERIMENT = "basic-chat"
```

In this example, we are using the *internal* URL of the MLflow server, which does **not** change for different RHOAI instances. You can use this URL only with applications running on the same OpenShift cluster.

d. Add code to configure the connection to the MLflow server, right before the code which gets your Kubernetes service account token.

```
import os
from openai import OpenAI
import mlflow

os.environ["MLFLOW_TRACKING_AUTH"]="kubernetes-namespaced"
mlflow.set_tracking_uri(MLFLOW_TRACKING_URL)
mlflow.set_experiment(MLFLOW_EXPERIMENT)

try:
    with open("/var/run/secrets/kubernetes.io/serviceaccount/token", "r") as f:
        AUTH_TOKEN = f.read().strip()
except FileNotFoundError:
    # Fallback if running outside a workbench or using a manually generated
cluster token
    AUTH_TOKEN = OCP_TOKEN
```

e. Add code to start MLflow autologging, right before the code which connects to the model server.

```
mlflow.openai.autolog()
client = OpenAI(
    base_url=BASE_URL,
    api_key=AUTH_TOKEN
```

```
)
```

2. Run your chat application to verify it continue working after changes.
    a. It is again a good time to save your code changes. In the top toolbar of your notebook, click the **disk** icon, having as tooltip "Save and create checkpoint".
    b. Ensure the second cell is selected and click the **play** icon in the top toolbar of your notebook.
    c. Hopefully, your output is again correct and useful, and probably have small differences in wording compared to your previous output. You can ignore warnings which suggest updating Jupyter and ipywidgets. More important, check that you see a message stating that a new experiment was created, similar to the following partial example output.

```
...
2026/07/01 19:47:59 INFO mlflow.tracking.fluent: Experiment with name 'basic-
chat' does not exist. Creating a new experiment.

RHEL (Red Hat Enterprise Linux) and CentOS are both Linux distributions, but
they have some key differences:
...
```

View and compare multiple traces

1. View the trace in the RHOAI dashboard.
    a. Switch to your browser tab on the RHOAI dashboard, which should be already in the Experiments page, and notice there is a new item, besides the `dummy experiment` created by the previous lab.
    b. Click the **basic-chat** experiment, which was created when you re-run your notebook after making changes.
    c. You will see a page displaying many statistics and graphics about the performance of your experiments. Each data point corresponds to one time you run your chat application.
    d. In the left pane of the Experiments page, without leaving the `basic-chat` experiments page, click **Traces**.
    e. You will see a page containing one trace per row. Each row displays a trace ID, its request, response, and some performance metrics.
    f. Click on either the trace id or its prompt to see a summary of the trace. Click on the links **Show 1 more** and similar on the input and output pane to expand the details of the inputs sent to the model server and the outputs received from it.

    From the information in a trace, it is possible to reproduce the request and compare its outputs between different attempts.

    g. Click the **x** icon to close the trace summary page and return to the list of traces from the `basic-chat` experiment.
2. Generate a second trace from the chat application.

We could repeat the same prompt multiple times, to verify variations in output and in performance metrics but, to make it more interesting, change the system prompt so you see a significant difference in the responses your application produces.

a. Switch to your Jupyter notebook and append more instructions to its system prompt:

```
input_text = """
What's the difference between RHEL and CentOS?
"""
system_instructions = """
You are a helpful AI assistant.
You are designed to answer questions in a concise and professional manner.
Answer with one to three sentences.
"""
```

b. Ensure the second cell is selected and click the **play** icon in the top toolbar of your notebook.

c. There should be no messages from MLflow, because it is using an existing experiment. And you should get a shorter response, which hopefully is still good, similar to the following example:

```
RHEL (Red Hat Enterprise Linux) and CentOS are both Linux distributions, but
they have different licensing and support models.
RHEL is a commercial distribution supported by Red Hat, while CentOS is a
community-supported distribution that mirrors RHEL.
CentOS is based on RHEL, but no longer receives security updates from Red Hat,
making RHEL the more secure choice.
```

If you see a button **Expand MLflow Trace**, do **not** click it. The visual integration between MLflow and RHOAI workbenches is not yet working with RHOAI 3.4.

3. View the additional traces in the RHOAI dashboard.

a. Switch to your RHOAI dashboards, which should be still in the Traces list of the `basic-chat` experiment. You may need to refresh your browser window to view the second trace.

b. Click the trace ID of the first item, which is the most recent trace, so it corresponds to the latest time you run your notebook.

c. Verify that the trace summary shows the latest, longer system prompt on its inputs, and the latest, shorter response on its outputs. Do **not** close the trace summary page.

Add assessments to your traces

1. Register a human assessment, simulating what a real-world chat application might do when a user clicks either a thumbs up or a thumbs down icon.

a. Click **Show assessments** to display the Assessments pane, to the right of the trace summary page.

b. Click **Add feedback** and select **Human feedback**.

c. Type `good response` as the Assessment Name. Leave the Data Type as `Boolean` and Value as `True` and click **Create**.

d. Click the **x** icon of the trace summary pane. There is also an **x** icon on the Assessments page, which is **not** the one you want.

e. Verify that the traces page of the `basic-chat` experiment now displays an Assessments column, and a summary of the distribution of values for the `good response` assessment among all traces.

f. Click the trace ID of the previous trace, from before you changed the system prompt. Add an human assessment using the same name but with a value of `False`. Close the summary page of that trace and see how the summary changes.

Summary

After completing these steps, you have executed a simple chat application with MLflow tracing enabled. You also compared traces from multiple runs of the chat application and recorded human assessments of its responses.

What's next

Now that you've added basic tracing to an agent, the next section explores how to trace MCP tool calls to capture more detailed information about agent behavior.

Lab: Trace MCP Tool Calls from an Agent

Objective

Deploy an MCP server and capture its tool invocations in MLflow traces.

Pending review.

Before you begin

You need access to the predeployed Red Hat OpenShift AI (RHOAI) instance provided in your lab environment, which the second lab already configured with a development instance of the MLflow server and an RHOAI workbench.

Ensure you have:

- Access credentials and the URL for the RHOAI dashboard
- A web browser to access the RHOAI dashboard
- Access credentials and the URL for the OpenShift web console

Instructions

You will deploy an MCP server which allows introspection of the state of an OpenShift cluster and its resources. Then, you will create a simple agent which uses that MCP server to assess the state of workloads running on your OpenShift cluster. Finally, you will interpret traces from the agent to

assess if it is able to make use of tools from the MCP server.

As you perform this activity, you will alternate between three browser tabs:

- The Red Hat OpenShift web console
- The Red Hat OpenShift AI (RHOAI) dashboard
- A Jupyter notebook from a RHOAI workbench.

Once you open one of these tabs, keep it open because you may return to it later.

Deploy and validate an MCP server

1. Deploy the MCP Server for Red Hat OpenShift with restricted privileges, by using its helm chart and the OpenShift Ecosystem Catalog.
   a. Open the OpenShift web console and log in using the `admin` user. In the left navigation menu, select **Ecosystem › Software Catalog**. Scroll down to find the **Type** heading and click **Helm charts**.
   b. Scroll up to the **All items** field and type `mcp`. Then click the entry **Red Hat OpenShift Mcp Server** and, on its details pane, click **Create**.
   c. In the **Create Helm Release** page, click the **Project** field, which probably displays the `default` project, and click **Create Project**. Type `mcp-servers` as the **Name** field and click **Create**.
   d. In the **Create Helm Release** page, delete all contents from its text editor and paste the following configuration, which gives the MCP server read-only access to all resources in the cluster, but disables external access.

```
ingress:
  enabled: false
rbac:
  extraClusterRoleBindings:
    - name: use-view-role
      roleRef:
        name: view
        external: true
```

   e. Click **Create**. You should be now in the **Workloads › Topology** page of the `mcp-servers` project, which displays a Helm icon for the MCP server deployment. Click the Helm icon to display its workload details page, to the right, and wait until its only pod is running.
2. Test the MCP server using the Gen AI studio
   a. Create a config map which provides the Gen AI studio with a list of available MCP servers.

      In the left navigation pane, select **Workloads › ConfigMaps** and, in the **Project** field at the top, select the `redhat-ods-applications` project.

   b. Click **Create ConfigMap** and, in the **Create Configmap** page, type `gen-ai-aa-mcp-servers` in

the **Name** field.

c. Type the following as the **Value** field and click **Create**.

```
{
      "url": "http://redhat-openshift-mcp-server.mcp-
servers.svc.cluster.local:8080/mcp",
      "description": "Red Hat OpenShift MCP server"
}
```

d. Verify that the Gen AI studio can connect to the MCP server.

Open the RHOAI dashboard and login as the `admin` user. In the left navigation pane, select **Gen AI studio › AI assets endpoints** and select the **MCP servers** tab. It should list the `OpenShift-MCP-Server`, as configured by the config map, with the `Active` status, which means the Gen AI studio was able to retrieve the tools list from the MCP server.

e. Test the MCP server tools using a playground.

In the left navigation pane, select **GenAI studio › Playground**. Then type `my` in the **Project** field and select **my-first-project**. Click **Create playground**. In the **Configure playground** form, let the model checked and click **Create**.

f. Wait while the Gen AI studio deploys a llama stack server for your playground. It displays a confirmation pop-up when done and enters the **Playground** page,

g. In the **Configure** pane of the playground, select the **MCP** tab and check the row with name `OpenShift-MCP-Server`. You should see a pop-up stating **Connection successful** and the number of tools provided by the MCP server. Click **Save** to select all tools.

h. In the right pane, click the `Send a message⋯` text and type a prompt which requires using the MCP server, for example:

```
Which pods are in my-first-model?
```

i. Type `Enter` to submit the prompt. Hopefully, you will get a correct response, which you can optionally verify by switching to the OpenShift web console and listing pods from the `my-first-model` project.

Import a simple agentic application from git

1. Import the Jupyter notebook for a simple agent.

a. In the left navigation pane, select **Projects** and, in the **Project** field, select **All projects**. Then type `my` and, in the list of projects, select **my-first-model**.

b. In the projects page for `my-first-model`, select the **Workbenches** tab. Click the **test-mlflow** workbench, which you created in the second lab, to open its jupyter lab in a new browser tab.

c. In the left toolbar of your Jupyter Lab, click the **Git Clone** icon.

d. In the **Clone a repo** popup, type `https://github.com/RedHatQuickCourses/rhoai-mlflow-samples.git` as the URI of the remote Git repository and click **Clone**.

e. In the file browser of your Jupyter Lab, enter the `/rhoai-mlflow-samples/traces` directory and then open the `basic-agent.ipnyb` notebook.

2. Review the code for a simple agent.

The explanations here are **not** intended to explain all code in the agentic application, but just its general structure, so you can later read its traces. The code snippets here enable learners to locate each section of code, if they wish to study and use it as a starting point for their agentic applications.

a. In its first cell, notice the use of `--extra-index-url` option for the `pip install` command. It is necessary because the RHOAI index does not include the `openai-agents` library. That way, pip falls back to the community index for any module not present in its primary index.

```
! pip install --extra-index-url https://pypi.org/simple openai openai-agents
mlflow[kubernetes]
```

b. In the second cell, notice the variables which set the URLs and connection parameters for the model server, the MCP server, and the MLFlow server. They are all internal URLs, which refer to their respective Kubernetes services.

```
BASE_URL = "http://llama-32-3b-instruct-predictor.my-first-
model.svc.cluster.local:8080/v1"
model_name = "llama-32-3b-instruct"

MCP_SERVER_URL = "http://redhat-openshift-mcp-server.mcp-
servers.svc.cluster.local:8080/mcp"

MLFLOW_TRACKING_URL = "https://mlflow.redhat-ods-
applications.svc.cluster.local:8443"
MLFLOW_EXPERIMENT = "basic-agent"
```

c. Then it sets a user prompt, which should require the MCP server, and a generic system prompt.

```
input_text = """
List the names of all pods in namespace 'my-first-model'
"""
system_instructions = """
You are a helpful AI assistant.
You are designed to answer questions in a concise and professional manner.
"""
```

d. So far, the code is very similar to previous examples but, after it gets the service account

token and enables MLflow tracing, the code to connect to the model and submit a prompt is *very* different from the previous chat example.

Differences start by creating an asynchronous client for the model server:

```
async def main():
    """Main async function to run chat with MCP tools"""

    # Configure custom OpenAI endpoint globally
    custom_client = AsyncOpenAI(
        base_url=BASE_URL,
        api_key=AUTH_TOKEN
    )
```

e. It also creates an asynchronous MCP client:

```
    async with MCPServerStreamableHttp(
        name="Remote MCP Server",
        params={
            "url": MCP_SERVER_URL,
            "timeout": 30,
        },
        cache_tools_list=True,
    ) as mcp_server:
```

f. The agent object refers, implicitly, to the global OpenAI endpoint configured earlier, and also refers explicitly to the MCP client and the system prompt.

```
        agent = Agent(
            name="Assistant",
            instructions=system_instructions,
            model=model_name,
            mcp_servers=[mcp_server],
        )
```

g. An asynchronous runner submits the prompt to the agent and displays its final response.

The agent loops on the responses received from the model, invoking MCP tools, and sending the output of tool calls back to the model.

This loop, inside the agent object, continues until the agent gets a response from the model which does **not** request another tool call.

```
        # Run the agent
        result = await Runner.run(agent, input_text)

        # Print the final output
```

```
        print("Assistant:", result.final_output)
```

h. The final piece of code enables running the asynchronous code from both a stand-alone application and a Jupyter notebook, which already sets an asynchronous runner for its cells.

```
        try:
            get_ipython()
            # Running in Jupyter - use await directly (top-level await is
supported)
            import nest_asyncio
            nest_asyncio.apply()
            asyncio.run(main())
        except NameError:
            # Not in Jupyter - use asyncio.run() normally
            asyncio.run(main())
```

View traces containing MCP tool calls

1. Run the agent to produce a trace.

   a. In the top toolbar from the notebook, click the **fast forward** icon, with the tooltip "Restart the kernel and run all cells".

   b. It may take a while to download all OpenAI and MLFlow libraries and their dependencies, in the first cell before it actually submits the prompt, in the second cell.

   If all goes well, you will get a correct response, similar to the following example:

```
Connecting to MCP server at http://redhat-openshift-mcp-server.mcp-
servers.svc.cluster.local:8080/mcp...

Found 14 tools from MCP server:

Processing query: List the names of all pods in namespace 'my-first-model'

Assistant: Based on the output, the names of all pods in the namespace 'my-
first-model' are:

1. llama-32-3b-instruct-predictor-6bd8fbdd49-28b9m
2. llama-32-3b-instruct-predictor-6bd8fbdd49-j9dxk
3. llama-32-3b-instruct-predictor-6bd8fbdd49-nkvn9
4. llama-32-3b-instruct-predictor-6bd8fbdd49-vvwgb
5. llama-32-3b-instruct-predictor-6bd8fbdd49-wbgxn
6. lsd-genai-playground-55555b9d99-78zqk
7. test-mlflow-0
```

   The output includes the number of tools from the MCP server as proof that the agent can connect to it, even if the response looks like it didn’t.

There is a higher chance of getting an incorrect response, or no response at all, compared to the previous example. The latter may happen because the agent received too many tool call requests from the model, and decided that the model is lost and cannot produce a final response.

2. View the trace in the RHOAI dashboard.

   a. Switch to the RHOAI dashboard and, in the left navigation pane, select **Develop & train › Experiments (MLflow)**. In the **Project** field, type `my` and select the **my-first-model** project. Then click the **basic-agent** experiment.

   b. In the left pane of the **Experiments** page for the `basic-agent` project, click **Traces**. You should see a trace whose request and response correspond to your notebook run. Click it.

   c. In the summary page for the trace, you will see a number of items labeled **AsyncCompletions**. Each of them corresponds to a round-trip between the agent and the model. If you are lucky, you will see two of them.

   d. Click the **expand/collapse** icon (>) from the first **AsyncCompletions**, but do **not** click the **AsyncCompletions** itself. You will see it contains **Inputs** and **Outputs**.

      The **Inputs** include the user and system prompts, a list of tools, and a number of OpenAI parameters which the sample agent didn't set explicitly, so they took their default values.

      The **Outputs:** include a request to call the tool named `pods_lists_in_namespace` with arguments `"namespace": "my-first-model"`

   e. Click the **expand/collapse** icon (>) from the second (or latest) **AsyncCompletions**.

      Notice its **Inputs** include everything you saw from the previous completions in the same trace, including prompts and lists of tools. But it adds the results of the tool call requested by the previous completion.

      It may be hard to read in the trace, but you can recognize similarities with the output of an `oc get pod -o wide` command.

      The **Outputs** should match the final response displayed by your notebook run.

   f. If you got more than two **AsyncCompletions** items, it might be because the model was not able to recognize that the tool call contains the information it needs, and requested a another tool call, maybe with different parameters, maybe of a different tool.

      For more complex prompts, it might be necessary to use the output of a tool call as parameters of another tool call, and many such rounds might be necessary. But you may find that, even with simpler prompts, a model might requests tool calls which make no sense.

   g. When you are satisfied that you understand the trace, click its **close** icon, that is, the **x** to the top right of the trace summary page, to return to the list of traces.

3. (Optional) You may switch to your Jupyter Lab and run the second cell again a few times. Each run produces its own trace, and they may take different numbers of completions to produce its

response.

a. Sometimes the responses will be correct, as in including the names of all pods and no extra pods, but vary on their wording.

b. Sometimes you may get an incorrect response, such as `This output indicates that there are no pods in the 'my-first-model' project.`

c. Notice that sometimes it uses the `pods_list_in_namespace` and other times it uses the `resources_list` tool. The latter usually leads to more spans to produce the final response.

d. Because the demo environment only supports a small model, with a small context window, you can easily devise a prompt which produces an error, such as listing all pods in the cluster. But you can also find that there are a number of useful prompts this small model can answer correctly and frequently.

e. You may also try subtle changes in the prompts, for example referring to "the my-first-model project" instead of to a "namespace", or trying to provide a better suited system prompt for an IT operations or a Kubernetes administrator persona.

Summary

After completing these steps, you have executed a simple agentic application which produces traces on the MLflow server in RHOAI. You also interpreted those traces to identify when the model requested tool calls, the specific data returned by the MCP server for each call, and how the data was send back from the agent to the model. Finally, you verified that the non-deterministic nature of large language models (LLMs) impacts their ability of requesting tool calls and using the returned data to produce a response.

What's next

Congratulations! You've completed this course and learned how to deploy MLFlow, collect traces from agentic applications, and visualize trace data. You can now apply these skills to troubleshoot and improve your own agentic applications.